# IT5008: Tutorial 7 — Triggers

Pratik Karmakar

School of Computing,
National University of Singapore

AY26/27 S1



# The Book Exchange Schema (Recap)

Same NUN Book Exchange scenario as before; the discussion questions use:

- **student**(name, email, year, department, graduate) — with `CHECK(graduate >= year)`
- **loan**(borrower, owner, book, copy, borrowed, returned) — `returned IS NULL` means *still out*

The preliminary questions use two small, separate schemas (employees; projects).

## New this tutorial

**Triggers**: PL/pgSQL functions that run automatically on `INSERT`/`UPDATE`/`DELETE` — to enforce constraints that `CHECK`, keys and foreign keys cannot express, or to *repair* a row instead of rejecting it.

# Trigger Cheat Sheet (Row-Level)

| | `BEFORE` | `AFTER` |
|---|---|---|
| Runs | before the row is written | after the row is written |
| `RETURN NEW` | write the row (possibly *modified*) | ignored |
| `RETURN NULL` | **silently skip** this row | ignored |
| `RAISE EXCEPTION` | abort the statement | abort the statement |
| Good for | reject or **fix up** one row | check the **resulting state** |

- `NEW` / `OLD`: the row after / before the change.
- `CREATE CONSTRAINT TRIGGER ... DEFERRABLE INITIALLY DEFERRED`: an `AFTER` trigger that runs at `COMMIT`, so a transaction may pass through invalid intermediate states.
- A trigger function must reach a `RETURN` (or raise) — even when the value is ignored.

# Preliminary: Triggers

*Not discussed in class — attempt before the tutorial, since later questions build on these.*

1. **Triggers.**
   (a) `Employees(eid, name)`, `Engineers(eid, skill)`, `Managers(eid, dept)`; both subclasses reference `Employees.eid`.
   No employee can be **both** an engineer and a manager.
   (b) `Engineers(eid, skill)`, `Projects(pid, max_hours)`, `Work(eid, pid, hour)`; `Work` references both.
   The total `hour` worked on a project cannot exceed its `max_hours`.

Enforce each with triggers.

# 1(a). Engineers Must Not Be Managers

**Schema:** Employees(eid, name), Engineers(eid, skill), Managers(eid, dept)
**FKs:** Engineers.eid → Employees.eid, Managers.eid → Employees.eid

A BEFORE trigger that **skips** the row (RETURN NULL) if the employee is already a manager.

```
1  CREATE OR REPLACE FUNCTION not_manager()
2  RETURNS TRIGGER AS $$
3  DECLARE
4    count NUMERIC;
5  BEGIN
6    SELECT COUNT(*) INTO count
7    FROM Managers
8    WHERE NEW.eid = Managers.eid;
9    IF count > 0 THEN
10     RETURN NULL;
11   ELSE
12     RETURN NEW;
13   END IF;
14 END;
15 $$ LANGUAGE plpgsql;
16
17 CREATE TRIGGER non_manager
18 BEFORE INSERT OR UPDATE ON Engineers
19 FOR EACH ROW EXECUTE FUNCTION not_manager();
```

# 1(a). . . . and Managers Must Not Be Engineers

The reference solution leaves this as an exercise. Without it, `INSERT INTO Managers` for an existing engineer still **succeeds**.

```
1  CREATE OR REPLACE FUNCTION not_engineer()
2  RETURNS TRIGGER AS $$
3  BEGIN
4    IF EXISTS (SELECT * FROM Engineers e WHERE e.eid = NEW.eid) THEN
5      RETURN NULL;
6    ELSE
7      RETURN NEW;
8    END IF;
9  END;
10 $$ LANGUAGE plpgsql;
11
12 CREATE TRIGGER non_engineer
13 BEFORE INSERT OR UPDATE ON Managers
14 FOR EACH ROW EXECUTE FUNCTION not_engineer();
```

- A constraint across two tables needs a trigger on **each** table that can break it.
- `RETURN NULL` rejects *silently* (`INSERT 0 0`); use `RAISE EXCEPTION` if the caller should know.

# 1(b). Project Hours Must Not Exceed max_hours

**Schema:** Engineers(eid, skill), Projects(pid, max_hours), Work(eid, pid, hour)
**FKs:** Work.eid → Engineers.eid, Work.pid → Projects.pid
**Rule:** for each project, SUM(hour) over its Work rows ≤ max_hours.

```
1  CREATE OR REPLACE FUNCTION not_exceeded()
2  RETURNS TRIGGER AS $$
3  BEGIN
4    IF EXISTS (
5      SELECT 1
6      FROM Projects p, Work w
7      WHERE p.pid = w.pid
8      GROUP BY p.pid
9      HAVING SUM(w.hour) > p.max_hours
10   ) THEN
11     RAISE EXCEPTION 'Hours exceeded!';
12   END IF;
13   RETURN NULL;
14 END;
15 $$ LANGUAGE plpgsql;
```

**Look for violations**: any project whose total hours exceed its limit.

# 1(b). The Triggers

```
1  CREATE CONSTRAINT TRIGGER project_limit
2  AFTER INSERT OR UPDATE ON Projects
3  DEFERRABLE INITIALLY DEFERRED
4  FOR EACH ROW EXECUTE FUNCTION not_exceeded();
5
6  CREATE CONSTRAINT TRIGGER work_limit
7  AFTER INSERT OR UPDATE ON Work
8  DEFERRABLE INITIALLY DEFERRED
9  FOR EACH ROW EXECUTE FUNCTION not_exceeded();
```

- `CONSTRAINT TRIGGER` + `DEFERRABLE INITIALLY DEFERRED`: the check runs at `COMMIT`.
- One trigger per table that can break the rule (next slide: why).

# 1(b). Why Two Deferred Triggers?

- **Both tables can break it:** adding/updating `Work` raises the total; updating `Projects` can lower `max_hours` below it.
- **Deferred to `COMMIT`:** the check looks at the *whole* database, so we let a transaction finish its changes before checking.

```
1  BEGIN;
2  INSERT INTO Projects VALUES (1, 10);
3  INSERT INTO Work VALUES (1, 1, 5);
4  COMMIT;                    -- OK: 5 <= 10
5
6  BEGIN;
7  INSERT INTO Projects VALUES (2, 10);
8  INSERT INTO Work VALUES (1, 2, 50);
9  COMMIT;                    -- ERROR: Hours exceeded!
10
11 UPDATE Projects SET max_hours = 4 WHERE pid = 1;
12                            -- ERROR: Hours exceeded!
```

# Discussion: Triggers on the Book Exchange

*To be discussed in class — attempt before the tutorial; you may be called on to present.*

2. **Triggers.**

   (a) Adeline Wong (`awong007@msn.com`) has already borrowed 6 books and returned none.
   New constraint: a student may borrow **at most 6 books at a time** — with 6 unreturned books, they cannot borrow another.

   (b) `student` has `CHECK(graduate >= year)`. We also want the graduation date to **not be in the future** — but instead of rejecting a bad value (as `CHECK(graduate <= CURRENT_DATE)` would), **change it** to the current date. Invoke on insert and update.

# 2(a). At Most 6 Books at a Time — Approach

Easiest with an `AFTER` trigger: let the row in, then **look for violating rows; if any, raise**.

```
1 IF EXISTS ( /* SQL query checking violating rows */ ) THEN
2   RAISE EXCEPTION '..';
3 END IF;
```

- **Violating row** here: a borrower with **more than 6** loans where `returned IS NULL`.
- Raising inside an `AFTER` trigger aborts the statement, so the offending loan is never kept.

# 2(a). The Trigger Function

```
1  CREATE OR REPLACE FUNCTION loan_limit()
2  RETURNS TRIGGER AS $$
3  DECLARE
4    violating_student RECORD;
5  BEGIN
6    -- Is there any student with more than 6 active loans?
7    SELECT l.borrower INTO violating_student
8    FROM loan l
9    WHERE l.returned ISNULL
10   GROUP BY l.borrower
11   HAVING COUNT(*) > 6;
12
13   IF violating_student IS NOT NULL THEN
14     RAISE EXCEPTION '% has borrowed more than 6 books',
15       violating_student;
16   ELSE
17     RETURN NEW;
18   END IF;
19 END;
20 $$ LANGUAGE plpgsql;
```

# 2(a). The Trigger

```
1 CREATE TRIGGER loan_limit_trigger
2 AFTER INSERT OR UPDATE ON loan
3 FOR EACH ROW EXECUTE FUNCTION loan_limit();
```

- **`> 6`, not `>= 6`:** the check runs *after* the new row is in. Adeline's existing 6 are fine; a 7th makes the count 7 and is rejected.
- **`UPDATE` too:** setting `returned` back to `NULL` re-opens a loan.
- Once she returns a book, she can borrow again.

To remove it:

```
1 DROP TRIGGER loan_limit_trigger ON loan;
2 DROP FUNCTION loan_limit();
3 -- or wipe everything:
4 DROP SCHEMA public CASCADE; CREATE SCHEMA public;
```

# 2(a). Alternative: a BEFORE Trigger

Check only the **new borrower**, *before* the row is written — so the limit is >= 6.

```
1  CREATE OR REPLACE FUNCTION loan_limit_before()
2  RETURNS TRIGGER AS $$
3  BEGIN
4    IF NEW.returned IS NULL AND (
5         SELECT COUNT(*) FROM loan l
6         WHERE l.borrower = NEW.borrower
7           AND l.returned IS NULL) >= 6
8    THEN
9      RAISE EXCEPTION '% already has 6 books', NEW.borrower;
10   END IF;
11   RETURN NEW;
12 END;
13 $$ LANGUAGE plpgsql;
14
15 CREATE TRIGGER loan_limit_before_trigger
16 BEFORE INSERT ON loan
17 FOR EACH ROW EXECUTE FUNCTION loan_limit_before();
```

**Off-by-one:** BEFORE sees the table *without* the new row, AFTER sees it *with* it.

# 2(b). No Future Graduation — Fix, Don't Reject

An `AFTER` trigger can only reject. To **change** the row, we need `BEFORE` and a modified `NEW`.

```
1  CREATE OR REPLACE FUNCTION graduation_date()
2  RETURNS TRIGGER AS $$
3  BEGIN
4    IF NEW.graduate > CURRENT_DATE THEN
5      NEW.graduate := CURRENT_DATE;
6    END IF;
7    RETURN NEW;
8  END;
9  $$ LANGUAGE plpgsql;
10
11 CREATE TRIGGER no_future_graduation
12 BEFORE INSERT OR UPDATE ON student
13 FOR EACH ROW EXECUTE FUNCTION graduation_date();
```

- `UPDATE student SET graduate = '2030-01-01' ...` stores today's date instead.
- **Still can fail** when `CURRENT_DATE < year`: the clamped date violates `CHECK(graduate >= year)`.

# Guidelines & Common Pitfalls

- **`BEFORE` to fix or skip a row, `AFTER` to check the result.** Only `BEFORE` can change `NEW` (2(b)).
- **Cover every table that can break the rule** — both subclasses in 1(a), both `Work` and `Projects` in 1(b).
- **Query for violations, not for valid rows.** `IF EXISTS(...) THEN RAISE` only works if the query finds the *bad* rows (1(b)).
- **Always reach a `RETURN`**, even in `AFTER` triggers (1(b)).
- **Mind the off-by-one:** `AFTER` counts the new row, `BEFORE` does not (2(a)).
- **Defer whole-database checks** with `CONSTRAINT TRIGGER ... DEFERRABLE INITIALLY DEFERRED` when a transaction needs several steps to become valid (1(b)).
- **Silent vs. loud:** `RETURN NULL` in `BEFORE` drops the row without an error; `RAISE EXCEPTION` aborts it.

Questions?
Drop a mail at: pratik.karmakar@u.nus.edu